Interview Preparation Guide: Amrita Das

Healthcare Data Science & Analytics Career Path

SECTION 1: YOUR PROFESSIONAL NARRATIVE

Who Are You? (30-second elevator pitch)

Your Answer: "I'm a Healthcare Data Analyst with 3+ years at Infosys specializing in claims processing and operational data. I've progressed from System Engineer to Senior Engineer to Technology Analyst, working with 10M+ healthcare claims annually. I'm skilled in SQL optimization, ETL pipelines, and batch processing, with hands-on experience in Python data science and machine learning. I've built three end-to-end production-ready ML projects—fraud detection, claim denial prediction, and data cleaning automation. I bridge the gap between operational analytics and machine learning, understanding both the healthcare domain and modern data science approaches. I'm excited to bring this combination of depth and breadth to solve complex healthcare analytics challenges."

Your Career Trajectory (How you've grown)

Your Answer: "I started as a System Engineer at Infosys, supporting healthcare claims operations—managing batch jobs, investigating data issues, and writing SQL queries. Over 3 years, I progressed to Senior Engineer, taking on more complex analytical work: root cause analysis, performance optimization, and dashboard design. Now as Technology Analyst, I'm architecting full data pipelines and applying machine learning to healthcare problems. The progression wasn't just title changes—each role gave me deeper understanding of healthcare data challenges. I moved from 'keeping systems running' to 'solving problems with analytics' to 'building AI solutions.' My three portfolio projects represent this evolution: starting with fraud detection (ML), then claim denial prediction (real-time ML), then data cleaning automation (AI/LLM integration). I'm not just a data person; I'm someone who understands healthcare operations deeply and applies technology strategically."

Why This Role? (Connect your background to the job)

Your Answer: "Your posting mentions [specific requirement from job description—analytics, fraud, claims, ML, etc.]. This is exactly my wheelhouse. I have 3 years of hands-on healthcare claims experience at Infosys working on real-world data challenges. I've solved the problems your team is likely facing: data quality issues, claim processing bottlenecks, fraud detection. And I've moved beyond just operational reporting—I'm now building ML systems to predict outcomes and automate decisions. The combination of domain knowledge, technical depth, and practical project experience makes me ready to contribute immediately while growing into more complex challenges. I'm genuinely excited about the intersection of healthcare and data science/AI."

SECTION 2: YOUR INFOSYS EXPERIENCE - DETAILED BREAKDOWN

Tell me about your work at Infosys

Your Answer (Comprehensive): "At Infosys, I worked on one of the largest healthcare claims processing systems in the world, handling 10M+ claims annually. My journey had three phases.

Phase 1 (System Engineer, Oct 2022 - Nov 2024): I supported healthcare claims operations—routine reporting, batch job monitoring, basic SQL analysis. I learned the healthcare domain: what claims are, denial patterns, processing workflows, regulatory requirements. I optimized SQL queries, managed Tidal scheduler workflows, and investigated production issues. This gave me deep operational understanding.

Phase 2 (Senior Engineer, Dec 2024 - Nov 2025): I took on more analytical responsibility. I designed data quality frameworks—automated validation checks that detected 500+ inconsistencies monthly. I performed exploratory data analysis (EDA) on claim denial patterns, identifying key risk factors. I led root cause investigations for batch failures, reducing incident recurrence by 35%. I designed Power BI dashboards that saved the operations team 20 hours/week. This phase was about moving from 'keeping systems running' to 'understanding problems deeply.'

Phase 3 (Technology Analyst, Dec 2025 - Present): Now I architect full data pipelines supporting ML workflows. I engineer end-to-end pipelines for claims analytics, prepare datasets for predictive models, develop claim-specific features, and validate model outputs for production. I achieved 40% SQL latency improvement through indexing and optimization. This phase is about using data science to drive decisions, not just report on them.

The progression shows I understand not just the technology, but the business: healthcare regulations, claim processing workflows, operational constraints, and where ML can add value. I'm not a pure data scientist parachuting in; I know the domain."

Explain your SQL optimization work (40% latency improvement)

Your Answer: "Query performance is critical in a high-volume claims system. When I started digging, I found several bottlenecks. First, many queries were full table scans on large claims tables (10M+ records). I identified missing indexes on frequently-filtered columns (claim_id, provider_id, claim_date) and created them. This alone cut some queries from 30 seconds to 8 seconds.

Second, I found unoptimized joins. Some queries joined 5+ tables without proper indexing or join order. I restructured these: joining smaller tables first, using proper join types (INNER vs LEFT), and adding covering indexes.

Third, I rewrote some calculations. Instead of calculating aggregations row-by-row, I used window functions (SUM OVER PARTITION BY) and CTEs (Common Table Expressions) to pre-aggregate data, then join once. This avoided redundant calculations.

Fourth, I analyzed execution plans using SQL explain tools to identify actual bottlenecks, not assumed ones. Sometimes a seemingly-complex query was actually fast; sometimes a simple-looking query was the culprit.

The result: queries that took 5+ minutes now complete in 1.5-2 minutes. For batch jobs running 1000s of queries nightly, this 40% improvement means the batch finishes earlier, reducing SLA risk and freeing compute resources. This is also good operational practice: faster queries mean less database load, cheaper cloud costs, and happier teams."

Tell me about your data quality framework

Your Answer: "Healthcare data quality is non-negotiable. A single wrong claim can cascade: affecting provider payments, patient records, and compliance. I designed a comprehensive validation framework.

Architecture: As data enters the system, automated checks run before it's used for reporting or analysis. Not after—before. This prevents garbage data from propagating.

The checks included: Schema validation (are columns the data type expected?), Completeness checks (are required fields present?), Referential integrity (does claim reference a valid provider and patient?), Business rule validation (is claim amount reasonable? Is admission date before discharge date?), and Outlier detection (flagging unusual but potentially valid values for manual review).

Implementation: These were SQL-based automated jobs running every batch cycle. If a check failed, the job would alert the team and quarantine suspicious data. Over a month, we detected and resolved 500+ inconsistencies. Without this, those bad records would have corrupted dashboards, analysis, and decisions.

Impact: We reduced data-related operational incidents by 35%. We gained confidence in the data, which meant analysts and ML engineers could focus on analysis, not data debugging. This framework also supported ML projects—clean data is the foundation of good models."

Walk me through your Power BI dashboard work

Your Answer: "I designed several operational dashboards in Power BI, the most impactful tracking claims processing status and denial metrics.

Content: The main dashboard showed real-time claim volume, processing status (claims in-flight vs. completed), denial rate trends, and detailed breakdowns by reason for denial, provider, and claim type. It included KPIs: total claims processed, average processing time, denial rate %.

Design: I kept it simple—leadership doesn't want complex analysis, they want the answer in 10 seconds. Key metrics at the top, trends over time, drill-down capability to detail. I used slicers for date range and claim type so users could filter instantly.

Data: It connected to SQL databases with automated daily refresh, so numbers were always current. No manual reporting—the dashboard pulled live data.

Impact: Before this, the operations manager spent ~20 hours/week manually compiling reports from various sources. Now they click a dashboard and see everything. That's 20 hours/week freed for actual strategic work. For claims operations, that's a big deal."

Tell me about the batch failures you investigated

Your Answer: "Batch processing is critical—if overnight jobs fail, the entire operation suffers. I regularly investigated failures.

Example 1: A job suddenly started failing at the reconciliation step. I checked logs (batch scheduler logs, database logs, application logs) and found a timeout error. Investigating further, a downstream table wasn't indexed properly, so join operations were slow. Fixed the index, job completed in 2 minutes instead of 30. Root cause: missing index. Prevention: added monitoring to alert if joins exceed X seconds.

Example 2: A job would fail intermittently (Tuesday mornings, for instance). Seemed random, but investigating the data, I noticed: on Mondays, claim volume spiked (end-of-week processing). This put pressure on the database. Tuesday morning jobs competed for resources and sometimes lost. Solution: resequenced the job dependencies so heavy-lifting happened when data volume was lower. Also added resource allocation tuning.

Example 3: A data transformation job failed when processing a new claim type the system hadn't seen before. The code assumed certain data patterns. Fixed: added validation to handle unexpected claim types gracefully instead of crashing.

Each investigation taught me: always check logs first, understand data volume patterns, think about system constraints (network, disk, memory), and build defensive code.

Impact: Reducing incident recurrence by 35% means fewer middle-of-the-night incidents, higher data quality, and more reliable systems. That builds trust."

SECTION 3: YOUR PORTFOLIO PROJECTS - DETAILED WALKTHROUGH

Project 1: Automated Fraud Claims Scoring (Your flagship project)

Why did you choose this project?

Your Answer: "Healthcare fraud is a $68+ billion annual problem in the US. It's impactful and relevant to my Infosys work—we process millions of claims annually, and fraud detection was a known pain point. Traditional rule-based systems generate too many false alarms. I wanted to build an ML system that could automatically score

provider risk based on behavior, catching fraud early without overwhelming investigators.

The project also challenged me technically: I had to solve unsupervised learning validation (how do you validate a fraud model without fraud labels?), build a production pipeline (not just a notebook), and explain model decisions to non-technical stakeholders. It combined domain knowledge, ML, engineering, and business thinking."

Walk me through the technical approach

Your Answer: "Approach: I used unsupervised anomaly detection with Isolation Forest. Here's why.

Problem: Fraud labels are rare, delayed, and unreliable. Training a supervised model on incomplete labels would overfit. Instead, I treated fraud as statistical deviation: providers who behave differently from peers are suspicious.

Data: I aggregated 10M+ claims by provider to calculate behavioral features: total claim volume, total billed amount, average claim amount per transaction, and unique beneficiary count. I flagged providers as 'high volume' or 'high cost' if they exceeded percentile thresholds.

Model: Isolation Forest detects anomalies by isolating outliers in feature space. It's fast (O(n log n)), doesn't assume Gaussian distributions, and gives explicit anomaly scores. Perfect for fraud detection.

Validation: Since there's no ground truth, I used stability metrics. I retrained the model 5 times with random variations and measured: score correlation (0.997-0.998—exceptional stability) and provider overlap (86-96%—same high-risk providers consistently flagged). This proved the model learned real patterns, not random noise.

Output: Daily batch scoring produces provider risk bands (Low/Medium/High). High-risk providers are investigated. Medium-risk are monitored. This helps investigators prioritize work."

What does 0.997-0.998 score correlation mean in plain English?

Your Answer: "Imagine you retrain the model five times. Each time, you get slightly different random initialization. A noisy model would produce very different scores each run—Provider A might be scored 0.7 one time, 0.2 the next. That's useless; investigators can't trust inconsistent alerts.

My model's correlation of 0.997-0.998 means: when I retrain, the scores are virtually identical. Provider A gets 0.7 every time. This tells me the model discovered real behavioral patterns, not random quirks.

Why it matters for production: You can't have a provider flagged as 'high fraud risk' one week and 'normal' the next. Investigators need consistent, stable alerts they can trust. The 0.997 correlation guarantees that. It's rare for unsupervised models to be this

stable—most are noisy. This stability is a key reason I'm confident deploying this to production."

Explain the 86-96% provider overlap metric

Your Answer: "This is the business version of score correlation. If I retrain the model five times, are the top 50 high-risk providers roughly the same each time?

Run 1: Providers A, B, C, D, E... (top 50) Run 2: Providers A, B, C, E, F... (top 50)

Overlap: How many appear in both lists? 90%.

Why it matters: If the top 50 churned randomly (only 50% overlap), investigators would lose confidence. If Provider X is truly high-risk due to behavioral anomalies, they should consistently appear in the top 50, not randomly shuffle in and out.

The 86-96% overlap proves the model makes reproducible, defensible decisions. Fraud investigators rely on this—they need stable alerts they can act on consistently."

How did you implement this in production?

Your Answer: "Production is not just code in a Jupyter notebook. I built:

Pipeline: Raw claims data → SQL feature engineering → Isolation Forest scoring → Risk band classification → Power BI dashboard. Everything is automated.

Shell Scripts: I created batch_scoring.sh that orchestrates the pipeline: checks prerequisites, runs feature engineering, loads the trained model, scores new data, validates output (did record counts change unexpectedly?), and loads results to the database. Error handling and logging at every step.

Batch Scheduling: Configured cron jobs (Linux) to run the script daily at 2 AM (off-peak). The script completes in ~15 minutes, new scores are ready by morning.

MLflow Tracking: Each training run logs hyperparameters, metrics (stability, overlap), and the trained model artifact. This creates a versioned history. If a new model performs worse, I can revert instantly.

Monitoring: I set up alerts: if the script fails, if score distributions shift abnormally, if high-risk provider overlap drops below 80%. The fraud team gets notified immediately.

Power BI Dashboard: Investigators log in to see the day's high-risk providers, claim counts, and financial exposure. They prioritize which to investigate. Every decision is logged.

This isn't a one-time analysis; it's a living system."

What would you improve if you had more time?

Your Answer: "Several enhancements:

1. **Real-time scoring: Currently batch daily. Real-time scoring would flag fraud immediately, reducing exposure time. Trade-off: more compute cost.**
2. **Explainability: Add SHAP values to show which features drove each provider's risk score. 'Your score of 0.8 is driven 60% by above-average claims, 40% by high volume.' Makes it less of a black box.**
3. **Feedback loop: Investigators confirm or reject alerts. Use this feedback to measure true positive rate over time and adjust the model. Currently it's one-way.**
4. **Drift monitoring: Track if score distributions shift or model performance degrades. Alert if retraining is needed.**
5. **Ensemble methods: Combine Isolation Forest with other anomaly detectors (LOF, autoencoders). Flag only if multiple algorithms agree. Higher confidence.**
6. **Integration: Connect to case management system. When an investigator opens a case, it's timestamped. Later, verify if the alert was correct. Learn and improve.**
7. **Cloud deployment: Containerize with Docker, deploy to Kubernetes or serverless (AWS Lambda). Auto-scale during peak periods. Better cost efficiency.**

These are nice-to-haves. The core system is production-ready now."

Project 2: Claim Denial Risk Prediction Pipeline

What was the business problem?

Your Answer: "Claim denials are expensive—when a claim is denied, the provider and patient incur costs. The insurer also spends money fighting appeals. The question: can we predict which claims will be denied before adjudication and intervene earlier?

I built a pipeline that predicts denial risk for incoming claims. High-risk claims can be reviewed immediately, potentially avoiding denial. Low-risk claims process normally. This prioritizes manual review work and reduces downstream denial costs."

Walk me through the technical approach

Your Answer: "Data: I engineered 10+ healthcare-specific features from raw claims. Examples: claim amount (high amounts are riskier), days between submission and adjudication (delays indicate complexity), diagnosis-procedure mismatch (procedures not matching diagnosis codes are risky), provider history (providers with high denial rates are higher risk), patient age (certain age groups have higher denial rates), etc.

Challenge: Data was highly imbalanced. 90% of claims are approved, 10% denied. A naive model trained on imbalanced data would predict 'approval' for everything and still be 90% accurate but useless. I used techniques to handle this: class weighting (penalize false negatives more), SMOTE resampling (oversample minority class), and threshold adjustment (predict 'denial' at a lower probability threshold).

Model: I trained a Logistic Regression model with class weighting. Why Logistic Regression? Interpretability—I can explain why a claim is predicted to be denied. Feature coefficients tell the story: 'High claim amount increases denial risk by X%.'

Training-Serving Consistency: I trained in Python/scikit-learn, serialized the model, and deployed it both in a batch job (daily scoring) and a FastAPI service (real-time scoring). Same model, same preprocessing, both inference paths produce identical predictions.

Deployment: Containerized with Docker, deployed to Render cloud. The API is accessible to the claims processing system. They submit claim data, get back denial risk score, and adjust workflow accordingly.

Impact: Claims likely to be denied are caught early, reducing downstream costs and improving approval efficiency."

Why Logistic Regression over more complex models?

Your Answer: "In production, simpler is often better. I also tested Random Forest and XGBoost. They had slightly higher accuracy, but:

1. **Interpretability: Logistic Regression coefficients directly tell you 'Feature X increases denial risk.' In healthcare, stakeholders need to understand model decisions. 'Why was this claim flagged as high-risk?' Logistic Regression answers clearly. XGBoost is a black box.**
2. **Training speed: Logistic Regression trains in seconds. XGBoost takes minutes. For daily retraining, speed matters.**
3. **Inference speed: Logistic Regression scores claims in milliseconds. Acceptable for real-time API. XGBoost is slower.**
4. **Fairness/Bias: Logistic Regression is easier to audit for bias. With XGBoost's complex interactions, bias is harder to detect and explain.**
5. **Regulation: Healthcare is regulated. Model decisions must be justifiable. Logistic Regression is easier to defend in an audit: 'Here's exactly why this claim was flagged.'**

The slight accuracy loss (maybe 92% vs 94%) is worth the gains in production readiness. This is a lesson: best model ≠ best production model."

Tell me about the FastAPI service

Your Answer: "FastAPI is a modern Python framework for building APIs quickly. I built an endpoint: POST /predict_denial_risk with claim data (amount, date, provider_id, diagnosis_codes, etc.), and it returns {claim_id, denial_risk_score, risk_category}.

Implementation: Load the trained model, define the input schema, add the prediction logic, and wrap it in FastAPI decorators. It's clean and fast.

Deployment: I Dockerized the FastAPI app (Dockerfile specifies Python version, dependencies, startup command). Deployed to Render, which auto-scales based on traffic. The claims system makes API calls when processing incoming claims.

Monitoring: The API logs every prediction request and response time. I track: throughput (claims/second), latency (milliseconds/prediction), error rate. If latency spikes, I auto-scale horizontally (add more instances).

Real-world use: A claim comes in. The system calls the API, gets a denial_risk_score. If high, it's routed to a senior reviewer. If low, auto-approval. This triages work efficiently.

This is production ML: not a notebook or demo, but a live service."

How did you ensure training-serving consistency?

Your Answer: "Training-serving skew is a hidden killer in ML. You train a model with clean data and certain preprocessing. When it goes to production, data is slightly different, preprocessing is slightly different, and suddenly the model performs poorly.

I prevented this by:

1. **Same preprocessing everywhere: Feature engineering code is shared across training and serving. I wrote a FeatureEngineer class that both training and inference pipelines import and use. No duplication, no divergence.**
2. **Model serialization: I saved the trained model using joblib in the training pipeline. Inference loads the exact same model artifact. No retraining or refitting happens in production.**
3. **Test pipeline: Before deploying to production, I ran the inference pipeline on historical data and compared predictions to training results. They matched perfectly.**
4. **Version tracking: The model file is versioned (model_v1.joblib, model_v2.joblib). The API config specifies which version to load. Easy to roll back if a new version has issues.**
5. **Input validation: The API validates input schema before prediction. Malformed data is rejected. Training data was validated similarly, so there's no mismatch in what the model sees.**

This discipline is the difference between a demo and a production system. You can't afford inconsistency."

Project 3: AI Data Cleaning Tool

What is this project about?

Your Answer: "Data cleaning is tedious and time-consuming—analysts spend 30-40% of time just cleaning data. I built an AI-powered tool that automates it. Users upload a CSV or Excel file, and the tool detects 15+ data quality issues: missing values, duplicates, inconsistent formatting, outliers, etc. The LLM generates fixing suggestions with explanations. Result: 95%+ improvement in dataset reliability and 80% reduction in manual preprocessing time.

This project showcases my ability to integrate modern AI (LLMs) with traditional data tools."

Walk me through the technical approach

Your Answer: "Stack: Python + Streamlit (web UI) + Groq (LLM API)

Frontend: Streamlit provides a simple, interactive web interface. Users click 'Upload File' → see detected issues → get suggested fixes → apply with one click. No coding required.

Detection: I implemented logic to detect:

- **Missing values: Columns with nulls**
- **Duplicates: Identical rows**
- **Inconsistent formatting: Mixed data types (1/1/2023 vs Jan 1, 2023)**
- **Outliers: Statistical anomalies (salary of $999,999 in a dataset of $50k average)**
- **Whitespace issues: Leading/trailing spaces**
- **Case inconsistencies: 'USA' vs 'usa'**
- **Data type mismatches: Numeric columns stored as strings**
- **And more...**

LLM integration: For each issue, I send details to Groq (a fast LLM API). Groq generates natural language suggestions: 'This column has 200 missing values (15% of data). Options: (1) Drop the column, (2) Impute with mean, (3) Impute with forward-fill. Choose based on your analysis needs.'

Execution: Users review suggestions and apply fixes. The tool regenerates the cleaned dataset.

Impact: 80% reduction in manual preprocessing time. What took an analyst 4 hours now takes 30 minutes (user uploads, reviews AI suggestions, applies)."

Why this project? What does it show?

Your Answer: "This project demonstrates:

1. **Modern AI integration: I didn't just use traditional ML. I integrated LLMs (Groq API) into a practical tool. This shows I'm current with AI trends.**

2. **Full-stack thinking: I didn't build an isolated model. I built a complete tool: data quality detection, LLM reasoning, interactive UI, file handling, error handling. It's production-ready.**
3. **User-centric design: The tool is built for non-technical users (analysts, business users). The UI is intuitive. Outputs are actionable. This shows I think about who uses my work and design accordingly.**
4. **Practical value: This directly solves a real problem. Every analyst deals with dirty data. A tool that cuts preprocessing by 80% is immediately valuable.**
5. **Skill breadth: SQL (checking data), Python (detection logic), LLM APIs (modern AI), Streamlit (UI), Docker (deployment). It shows I'm not siloed in one skill."**

Deployment and usage

Your Answer: "I containerized it with Docker and deployed it to a cloud service. The tool is accessible via a web link. Users can upload files, see results, and download cleaned data. It's self-service—no data science team needed to run it.

Monitoring: I track usage—how many files uploaded, which issues are most common, user satisfaction. This tells me where to focus improvements.

This tool could be deployed internally in a company or even offered as a SaaS product. It's that polished."

SECTION 4: YOUR TECHNICAL SKILLS - DEEP DIVES

SQL Skills

Your Answer (if asked to explain SQL expertise): "I'm proficient in enterprise SQL—not just SELECT-FROM-WHERE. I use:

Aggregations and Window Functions: SUM() OVER PARTITION BY to calculate running totals, ranking providers by claim count, calculating percentiles. These are essential for healthcare analytics.

CTEs (Common Table Expressions): Decompose complex queries into readable steps. WITH temp_table AS (...) SELECT ... simplifies logic.

Joins: INNER, LEFT, RIGHT, FULL OUTER, CROSS. I understand join implications—LEFT JOIN might return duplicates if not careful. I design joins to minimize rows early.

Indexing: Not just SELECT; understand why queries are slow. Use EXPLAIN to analyze execution plans. Create indexes on frequently-filtered columns. Monitor index fragmentation.

Stored Procedures: Write reusable SQL logic encapsulated in procedures. Call from Python or batch jobs.

Query Optimization: Rewrite slow queries. Example: instead of calculating aggregations for every row, pre-aggregate with GROUP BY, then join. 10x faster.

Performance Tuning: Understand database resources (CPU, memory, I/O). Monitor slow query logs. Analyze table statistics. Adjust buffer pools and caching.

In my Infosys role, I took queries running 5+ minutes down to 1.5 minutes. That's tangible impact."

Python Skills

Your Answer: "I use Python for data manipulation, ML, and automation.

Data Manipulation: Pandas is my bread and butter. Groupby, merge, pivot tables, string operations. I handle messy healthcare data: dates in different formats, names with special characters, missing values.

Numerical Computing: NumPy for vectorized operations, faster than loops. Healthcare datasets are large; NumPy operations are orders of magnitude faster.

Machine Learning: Scikit-learn for traditional models (Logistic Regression, Isolation Forest, Random Forest). I train, validate, and deploy models. I understand model evaluation metrics: precision, recall, F1, AUC-ROC (crucial for imbalanced data like claim denials).

Feature Engineering: I don't just use raw data. I engineer features: ratios (claim_amount / peer_average_claim), temporal patterns (days since last claim), interactions (high volume AND high cost). Feature engineering is often more important than the model choice.

Scripting: I automate tasks with Python scripts. Database queries, data transformations, file handling, error logging. These scripts run unattended in batch jobs.

APIs: FastAPI for building services. I've built a real-time prediction API deployed to the cloud.

LLM Integration: I've integrated Groq API into a Streamlit application. I understand prompt engineering and API usage patterns.

Testing and Debugging: I write unit tests. I debug using logging and print statements. I handle exceptions gracefully.

My Python is practical, production-oriented, not just academic."

Power BI Skills

Your Answer: "I'm certified as a Power BI Developer (Infosys certification). I use it to translate data into actionable insights for non-technical stakeholders.

Data Connections: Connect to SQL databases, Excel, cloud sources. Set up automated daily refresh.

Data Modeling: Create relationships between tables, calculate columns, measures using DAX (Data Analysis Expressions). DAX formulas are like Excel on steroids—calculate complex metrics like 'Year-over-Year growth' or 'Cumulative denial rate.'

Visualizations: I know when to use which chart. Line chart for trends, bar chart for comparisons, scatter plot for relationships, KPI cards for metrics. I avoid chart junk—every visual element serves a purpose.

Dashboards: Design multi-page dashboards with clear information hierarchy. KPIs at top, detailed data below, drill-down capability. Slicers for filtering. Interactive, not static.

User Experience: I keep dashboards simple. Leadership doesn't want to spend time learning—they want answers in 10 seconds. Color coding (red for alert, green for good) helps at a glance.

Performance: Optimize dashboards so they load quickly. Use aggregated tables instead of raw data. Import data instead of live query when possible.

Sharing and Security: Set up role-based security so users see only authorized data. Share dashboards via Power BI Service (cloud).

I've built dashboards that replaced 20+ hours/week of manual reporting. That's impact."

MLflow and Experiment Tracking

Your Answer: "MLflow is an open-source platform for ML lifecycle management. In my fraud detection project, I integrated it to track experiments.

Usage: After training a model, I log:

- **Hyperparameters: n_estimators=100, max_depth=10, contamination=0.02**
- **Metrics: score_correlation=0.9978, provider_overlap=0.92, training_time=45 seconds**
- **Artifacts: The trained model file (joblib), feature preprocessing (pickle)**
- **Tags: experiment_name='fraud_detection_v3', data_date='2025-01-30'**

Benefit: I can compare experiments. Run 5 different model configurations, log all of them to MLflow, then compare which had best stability metrics. This creates a versioned, auditable history.

Model Registry: Mark certain model versions as 'production' or 'staging'. Production is what's deployed. If a new version doesn't perform well, revert instantly by changing the registry tag.

Monitoring: Track metrics over time. Plot score_correlation across retraining runs. If it suddenly drops, that's a signal something's wrong.

This is operationalizing ML: not a one-time training, but continuous tracking and improvement."

Batch Processing and Scheduling

Your Answer: "Batch processing is scheduling jobs to run unattended, typically nightly.

Tidal Scheduler (at Infosys): Enterprise batch scheduler. Define jobs, dependencies, and schedules. If Job A finishes, Job B starts automatically. If Job B fails, notify ops, don't run Job C. This orchestrates complex workflows.

Shell Scripts (my projects): I write bash scripts that handle a multi-step pipeline. Pre-checks (is the database available?), run Python feature engineering, run model scoring, validate output, load to database, send alerts if anything fails. The script is idempotent: running it multiple times is safe, won't create duplicates.

Cron Jobs (Linux): Schedule shell scripts to run daily at specific times. Example: 0 2 * * * /scripts/batch_scoring.sh (run at 2 AM every day). Logs go to a date-stamped file for later review.

Windows Task Scheduler: Equivalent on Windows.

Error Handling and Recovery: Batch jobs should fail gracefully. Log errors, alert humans, retry if transient. Don't cascade failures—if one step fails, don't proceed to the next.

This is the operational side of data science: the code that runs in the background, reliably, every day."

Docker and Containerization (Basic)

Your Answer: "Docker containerizes an application so it runs the same everywhere: local machine, test server, production. No 'works on my computer, breaks in production' problems.

Dockerfile: Specifies the base image (Python 3.9), installs dependencies (pip install -r requirements.txt), copies code, and specifies the startup command. Example:

FROM python:3.9 COPY requirements.txt . RUN pip install -r requirements.txt COPY . . CMD ['python', 'app.py']

Build: docker build -t my-app . Run: docker run my-app

Deployment: Push the image to a container registry (Docker Hub, ECR), then deploy to a cloud platform (AWS ECS, Google Cloud Run, Azure Container Instances). The container runs identically across environments.

I've containerized my Claim Denial API and Data Cleaning Tool. This is modern deployment practice."

SECTION 5: HEALTHCARE DOMAIN KNOWLEDGE

Explain healthcare claims processing

Your Answer: "Here's how claims flow: A patient visits a provider (hospital, doctor). The provider submits a claim to the insurance company, requesting payment. The claim includes: patient info, provider info, service dates, procedures, diagnosis codes (ICD-10), billed amount, etc.

The insurer's system processes the claim. Steps include:

1. **Eligibility check: Is the patient covered?**
2. **Benefits verification: What does their plan cover?**
3. **Network validation: Is the provider in-network?**
4. **Medical policy: Does this procedure require pre-authorization?**
5. **Fraud/compliance: Does this claim look suspicious?**
6. **Pricing: What should be paid based on fee schedule?**
7. **Adjudication: Approve, deny, or request more info.**

Possible outcomes:

- **Approved: Payment issued.**
- **Denied: Patient and provider notified.**
- **Pended: More info requested.**
- **Adjusted: Approved for lower amount.**

Common denial reasons: Service not covered, pre-auth required, out of network, duplicate claim, medical necessity not met.

My ML projects fit here: fraud detection (step 5) and denial prediction (step 7)."

What is claim denial and why does it matter?

Your Answer: "A claim denial is when the insurer refuses to pay. The provider doesn't get reimbursed, the patient might be billed, and there are appeal costs on both sides.

Why it matters:

- **Financial: Millions in denied claims = massive revenue loss for providers. Appeals cost time and money.**
- **Patient impact: Patients might be stuck with bills. Frustration with both provider and insurer.**
- **Operational: Denials create rework: patients call, providers file appeals, insurer re-reviews. Everyone's time is wasted.**

My denial prediction project predicts which claims will be denied before adjudication. High-risk claims can be reviewed immediately, potentially overturned before denial, saving everyone time and money.

Prevention is better than treatment (appeals)."

What is healthcare fraud and why is it a problem?

Your Answer: "Healthcare fraud is when providers, patients, or suppliers knowingly deceive insurers for financial gain. Examples:

- **Provider overbilling: Charging for services not provided.**
- **Upcoding: Billing for expensive procedure instead of cheaper one actually performed.**
- **Unnecessary services: Recommending or billing for unneeded tests/procedures.**
- **False claims: Billing for patients who don't exist.**
- **Network fraud: Provider billing at in-network rates while providing out-of-network care.**

Why it's a problem:

- **Financial: $68+ billion annually in the US. That's real money that could go to legitimate care.**
- **Patient impact: Fraudulent billing inflates insurance premiums, affecting everyone.**
- **System integrity: Fraud erodes trust in healthcare.**

My fraud detection project identifies providers with suspicious billing behavior—high volume, high cost, or patterns unusual for their specialty. These are red flags for investigation.

This is high-impact work."

SECTION 6: QUESTIONS ABOUT YOUR GROWTH & MOTIVATION

Why did you transition from operations to data science?

Your Answer: "At Infosys, I started in operations—keeping systems running, managing batch jobs, basic SQL reporting. It was valuable, but I wanted to go deeper. Operational work answers 'What happened?' (dashboards, reports). I wanted to answer 'Why did it happen? What will happen next? How do we improve?'

That's where data science comes in. I started learning Python, reading about ML, and building projects. My three portfolio projects—fraud detection, denial prediction, data cleaning automation—were self-driven. They weren't assigned; I pursued them because I was passionate about solving problems with ML.

I didn't abandon operations knowledge; I built on it. I understand healthcare claims deeply, so I engineer better features. I understand batch processing, so I build production-grade pipelines. I'm not a pure data scientist; I'm a data scientist with operational maturity. That combination is rare and valuable."

How do you stay current with tech?

Your Answer: "Healthcare and data science evolve fast. I stay current through:

1. **Learning by doing: I build projects with new tools. FastAPI, Groq LLM, Streamlit—I learn by building. Not just reading.**
2. **Reading: Blogs (Medium, Towards Data Science), papers, documentation. Not passively; I take notes and try concepts.**
3. **Online courses: I completed 'Career Essentials in Data Analysis' from Microsoft & LinkedIn. I pursue Infosys certifications (Power BI, Azure).**
4. **Communities: I engage on GitHub, follow data science discussions, participate in discussions (not lurk).**
5. **Experimentation: I try new approaches. If a new algorithm sounds interesting, I test it on a project.**
6. **Feedback loops: I ask colleagues, mentors, and users for feedback. What could I do better?**

This discipline keeps me sharp and relevant."

Describe a time you faced a technical challenge

Your Answer: "Challenge: In the fraud detection project, I struggled with validating an unsupervised model. Supervised models are easy—you have ground truth labels, compare predictions to labels, compute accuracy. Unsupervised? No labels. How do you validate?

I researched extensively: academic papers on anomaly detection validation, industry blogs on fraud detection, talking to fraud investigators. I learned that stability is key: if the model produces consistent, reproducible scores across retraining runs, that's evidence of learning real patterns.

So I devised stability metrics: score correlation (0.997-0.998 across retraining) and provider overlap (86-96% of top-risk providers consistent). I validated these made sense: talked to fraud teams, confirmed that stable alerts are what they need.

Result: Not only did I solve the validation problem, but the stability metrics became the key evidence that the model was production-ready.

Lesson: Sometimes the problem isn't technical; it's conceptual. Once I framed it correctly (stability = validation), the solution became clear. I also learned to talk to domain experts, not just data. That brings practical grounding to technical work."

Tell me about a time you had to work with stakeholders

Your Answer: "At Infosys, I designed a Power BI dashboard for the operations team. Initially, I showed them a complex dashboard with 20+ metrics, drill-down to claim-level detail, etc. The response: 'Too much. We need a one-page summary we can glance at in 10 seconds.'

That feedback stung, but it was right. I iterated: kept only the key metrics (claim volume, processing status, denial rate), designed for at-a-glance understanding, added simple slicers for filtering. I showed a mock-up, got feedback, refined.

The final dashboard was simpler but much more valuable. The ops manager could log in, see the day's status immediately, make decisions. No time wasted digging for data.

Lesson: Don't build what you think is technically impressive. Build what users actually need. Talk to them early and often. Iterate based on feedback. Collaboration > Technical Perfection."

SECTION 7: CERTIFICATIONS & AWARDS

Tell me about your certifications

Your Answer: "I have several Infosys certifications validating my skills:

Power BI Developer: Demonstrates proficiency in dashboard design, data modeling, DAX formulas. This certification required hands-on exams, not just multiple choice.

Azure Integration Services Developer: Shows I understand cloud platforms, specifically Azure. I've deployed applications to Azure, though I'm still gaining depth here (I marked myself 'beginner' for cloud on my resume—honesty matters).

Global Agile Developer: Certifies I understand Agile methodology, Scrum practices, collaborative development. Infosys operates on Agile, so this is relevant.

Career Essentials in Data Analysis (Microsoft & LinkedIn): This is current and shows I'm invested in structured learning. Not self-taught alone; I pursue formal credentials too.

These certifications validate specific skills and show commitment to growth."

Tell me about the Insta Award

Your Answer: "Insta Award is Infosys's recognition for outstanding contributions. I received it for my work on data quality frameworks and batch failure investigations. My contributions had measurable impact: reduced data-related incidents by 35%, cut manual reporting by 20 hours/week, and improved SLA adherence.

The award wasn't just about technical skill; it was about business impact. I solved real problems that mattered to the organization. That recognition validates that I deliver value beyond just coding."

SECTION 8: INTERVIEW QUESTIONS ABOUT YOUR EDUCATION

Tell me about your Masters (MCA)

Your Answer: "I completed my Master of Computer Applications (MCA) at Trident Academy of Technology with a 9.05/10 GPA. This was a rigorous 2-year program covering computer science fundamentals and advanced topics.

Relevant coursework:

- **Databases: SQL, database design, normalization. Foundation for my current data work.**
- **Data Structures & Algorithms: Essential CS fundamentals. Understanding these helps me write efficient code, debug complex systems, and think algorithmically.**
- **Machine Learning: Early exposure to ML concepts, algorithms, and applications. I build on this foundation daily.**
- **Data Mining: Precursor to modern data science. Understanding data extraction and pattern discovery.**
- **Operating Systems, Networks: Systems knowledge. Useful when debugging infrastructure issues, understanding how batch systems work.**

The MCA was rigorous, especially algorithms and OS courses. It gave me a strong CS foundation that informs my work today. You can't do good data science without understanding fundamentals like time/space complexity, data structures, and systems concepts."

SECTION 9: BEHAVIORAL & SITUATIONAL QUESTIONS

Describe your proudest achievement at Infosys

Your Answer: "I'm most proud of the data quality framework I built. Here's why: It solved a pervasive problem (bad data) systematically. It prevented 500+ monthly data errors before they could corrupt analysis. It gave the team confidence in data quality, freeing analysts from debugging and toward real analysis.

But beyond the technical achievement, I'm proud of the impact. The ops team needed reliable data; I delivered it. Investigators needed to trust the data feeding their dashboards; they could now. This is what I love about data work: the technical challenge plus the human impact.

Measured impact: 35% reduction in data-related incidents, and the team directly told me the framework made their work easier. That's satisfaction."

Tell me about a time you disagreed with a colleague

Your Answer: "At one point, I proposed reengineering a batch job to run during off-peak hours (reducing database load and cost). A colleague argued it wasn't necessary—the job was 'fast enough.' I understood their caution: changes to batch jobs are risky; if something breaks, claims processing breaks.

I didn't push back emotionally. Instead, I showed data: query execution times, database load metrics, cost breakdowns. I demonstrated that my change was safe: I'd tested it on staging data, the logic was identical, just timing changed. I also had a rollback plan: if anything went wrong, revert to the old schedule instantly.

Seeing the data, they agreed. We implemented the change, saved the company ~$2k/month on compute, and reduced database load stress.

Lesson: Disagreements are normal. The way to resolve them: data, not opinion. Also, understand the other person's concern (they feared breaking production) and address it directly (I had a rollback plan)."

Tell me about a failure and what you learned

Your Answer: "Early in my Infosys career, I wrote a SQL query for a batch job without fully understanding the downstream impact. The query was correct, but slow. When the batch ran, it caused database locks that affected other processes. The team had to manually stop the job—incident!

At first, I was defensive. 'The query is correct; the database is slow.' But I realized I was wrong. In a complex system, 'correct' isn't enough—it has to work within system constraints. I learned to:

1. **Always test on realistic data volumes and infrastructure, not my local machine.**
2. **Understand the full system: what other jobs depend on this? What resources are shared?**
3. **Get code reviewed before deployment. Another pair of eyes catches issues.**

4. **Have monitoring: if a query suddenly runs slow, alert humans before it breaks things.**

This incident was small, but the lesson stuck. Systems thinking is essential. Since then, I've built safer, more aware code."

Where do you see yourself in 3-5 years?

Your Answer: "In 3-5 years, I want to progress from building individual projects to leading teams of data scientists and engineers. I want to:

- **Own end-to-end data science initiatives: scoping problems, building solutions, deploying to production, measuring impact.**
- **Mentor junior data scientists: sharing what I've learned, helping them grow.**
- **Influence data strategy: not just executing projects, but deciding which projects matter most.**
- **Go deeper into specialized areas: maybe healthcare AI/ML, advanced time-series forecasting, real-time streaming systems.**
- **Contribute to the community: writing about lessons learned, open-sourcing useful tools, maybe speaking at conferences.**

I see myself as a senior data scientist or tech lead: someone the team turns to for hard problems, someone who balances technical rigor with business pragmatism, someone invested in growing the team."

SECTION 10: QUESTIONS TO ASK THE INTERVIEWER

These show strategic thinking:

1. **"Tell me about the biggest data/analytics challenge your team faces. What's the bottleneck?"**
2. **"How does your organization measure success for analytics/ML projects? What matters most—accuracy, speed, cost, business impact?"**
3. **"What's the current data infrastructure? Do you have good data quality, governance, and security practices?"**
4. **"How do you handle model deployment and monitoring? Is there an established ML ops practice?"**
5. **"What's the biggest pain point with your current systems—is it data quality, model performance, operational complexity?"**
6. **"Who are the stakeholders I'd be working with? Clinicians, ops teams, executives?"**

7. **"What does the career path look like for a data scientist here? How do people progress to senior/lead roles?"**
8. **"What drew you to this role / team? What do you enjoy most about working here?"**
9. **"If I join, what would success look like in the first 90 days?"**
10. **"How does your team handle failures or model issues? What's the blame culture like?" (This reveals company values.)**

SECTION 11: TECHNICAL DEEP DIVES FOR TRICKY QUESTIONS

Explain how you'd approach a new problem (Method)

Your Answer: "Step 1: Clarify the problem. What's the business question? Who's the stakeholder? What's the expected impact? Is it a real problem or a nice-to-have?

Step 2: Understand the data. What data exists? Quality? Coverage? Size? Can I access it easily?

Step 3: Explore. EDA. What patterns exist? Correlations? Outliers? Missing data?

Step 4: Baseline. What's the simplest solution? Rule-based approach? Simple model? This tells me what ML needs to beat.

Step 5: Model selection. Based on exploration, what approach makes sense? Classification, regression, anomaly detection? Supervised or unsupervised? Why?

Step 6: Build and validate. Train, cross-validate, test. Measure performance. Make sure it actually solves the problem, not just optimizes a metric.

Step 7: Iterate. Refine features, try other models, get feedback from stakeholders.

Step 8: Deploy and monitor. It's not done until it's live and performing well in production.

This is systematic and grounded in reality, not just technical optimization."

How would you explain a ML model to non-technical stakeholders?

Your Answer: "I avoid jargon. No 'Isolation Forest,' 'anomaly scores,' 'feature engineering.'

Instead: 'We built a system that learns normal provider behavior: how many claims they submit, how much they charge, who they see. Then, we flag providers who behave differently—maybe they suddenly doubled claims or charge 10x more than peers. Those differences could indicate fraud. The system scores every provider daily, flags high-risk ones, and investigators prioritize.'

I use analogies: 'It's like a security system. You learn the pattern of normal traffic. Then, when something unusual happens—a break-in attempt—it alerts you. Our model does the same, but for healthcare claims.'

I relate to their world: 'You've been investigating claims for 10 years. You have intuition for what's suspicious. Our model learned similar patterns from data. We flag the same providers you'd suspect, but automatically, without human fatigue.'

I avoid false precision: 'This model is 95% accurate' (misleading in unsupervised settings). Instead: 'This model is stable—it flags the same providers consistently, and manual review confirms they're actually suspicious.'

Story > Math."

What if someone asks "What are you weaknesses?"

Your Answer: "Honesty with a growth arc. Not fake weaknesses ('I'm a perfectionist'), and not critical weaknesses that disqualify you.

Example: 'Early on, I was more comfortable with SQL and batch processing than Python and ML. I addressed this deliberately—built projects, took courses, learned by doing. I'm now proficient in ML and Python, and I continue to deepen. I know there's always more to learn in AI/LLMs; it's a fast-moving field, but I'm committed to staying current.'

Another: 'I tend to dive deep into technical problems, sometimes losing sight of the bigger context. I've learned to step back and ask: "Does this actually matter to the business? Am I optimizing the right thing?" I now involve stakeholders early, which keeps me grounded.'

The pattern: Acknowledge, take action, learn. Shows self-awareness and growth mindset."

How do you approach learning new tools (Git, Docker, etc.)?

Your Answer: "I learn by doing, not passively. When I needed Git for version control, I didn't just read docs. I set up a repo, made commits, branches, merges, resolved conflicts. Hands-on experience sticks.

Docker: I containerized my FastAPI app. Wrote a Dockerfile, built an image, ran locally, deployed to cloud. That's learning.

For LLM APIs: I integrated Groq API into my data cleaning tool. Experimented with different prompts, understood rate limits and error handling. Real-world usage beats tutorials.

For tools I haven't used yet (e.g., Kubernetes): I'll learn the same way. Start with a simple project, gradually increase complexity. Documentation + practice.

I'm not afraid of new tools. I see them as problems to solve, and I enjoy the challenge. This openness is important in a fast-moving field."

SECTION 12: FINAL CONFIDENCE ANCHORS

Your Core Value Proposition

In an interview, you are:

1. **A healthcare data specialist with 3+ years of real-world experience**
2. **A bridge between operations and data science**
3. **A full-stack data scientist: problem definition → data engineering → ML modeling → deployment → monitoring**
4. **Someone who understands both technical rigor and business pragmatism**
5. **A self-driven learner: you build projects, not just work assigned tasks**

Your Key Achievements to Highlight

1. **40% SQL latency improvement through optimization (tangible, quantified)**
2. **500+ monthly data inconsistencies prevented through quality framework (scalable impact)**
3. **Fraud detection model with 0.997-0.998 stability (exceptional validation)**
4. **86-96% provider overlap (proof of production-ready stability)**
5. **20 hours/week saved in manual reporting (operational impact)**
6. **Three end-to-end portfolio projects (breadth and depth)**

Your Unique Combination

You're not just a data scientist (many are). You're a data scientist who:

- **Understands healthcare operations deeply**
- **Can optimize SQL queries and design batch systems**
- **Can explain models to non-technical stakeholders**
- **Can code full-stack (backend, API, frontend)**
- **Has deployed real systems to production**
- **Measures success by business impact, not just technical metrics**

Red Flags to Avoid

- **Don't undersell your Infosys work. You've had real, substantial responsibilities.**

- **Don't oversell your cloud experience if you marked it beginner. Be honest.**
- **Don't claim expertise in tools you've only tinkered with.**
- **Don't focus solely on technical complexity; always connect to business value.**
- **Don't badmouth Infosys or previous roles. You learned valuable skills there.**

Pre-Interview Checklist

- **Review the job description. What skills do they emphasize? Prepare to discuss those.**
- **Research the company. What data/ML challenges do they likely face? (Check their blog, LinkedIn, news.)**
- **Prepare your portfolio project descriptions. You should be able to explain each in 2 minutes naturally.**
- **Think of 2-3 specific examples: challenges you solved, decisions you made, feedback you received.**
- **Prepare questions to ask. Shows you're serious and strategic.**
- **Get a good night's sleep. You're sharp, confident, and ready.**

SECTION 13: SPECIFIC COMPANY/ROLE SCENARIOS

If the role is Healthcare Analytics

Key Talking Points:

- **3 years healthcare claims experience at Infosys**
- **Understand claims processing workflows deeply**
- **Built fraud detection and denial prediction systems**
- **Power BI dashboard experience**
- **SQL and ETL pipelines**
- **Bridge between operational needs and analytics**

If the role is Data Science/ML

Key Talking Points:

- **End-to-end ML pipeline architecture**
- **Feature engineering expertise (10+ features in denial prediction)**
- **Unsupervised learning validation (stability metrics)**

- **Model deployment and monitoring**
- **Production-grade systems, not just notebooks**
- **Three portfolio projects showing breadth**

If the role is Data Engineering

Key Talking Points:

- **SQL optimization (40% improvement)**
- **ETL pipeline design**
- **Batch processing with shell scripts and scheduling**
- **Data quality frameworks**
- **Working with large datasets (10M+ claims)**
- **Building pipelines for downstream analytics/ML**

If the role is MLOps/ML Engineering

Key Talking Points:

- **Model versioning with MLflow**
- **Docker containerization**
- **Automated batch scoring pipelines**
- **API deployment (FastAPI)**
- **Monitoring and reliability**
- **Training-serving consistency**

FINAL PRE-INTERVIEW PEP TALK

You've built real, production-grade systems. You understand healthcare domain deeply. You've progressed from support roles to analytical/ML roles through self-directed learning. You have certifications, awards, and portfolio projects that prove capability.

In the interview, you're not asking for a chance. You're demonstrating why you're the right person for the job. You've solved problems they likely face: data quality, fraud detection, claim processing analytics. You've deployed systems to production, not just prototyped. You measure success by business impact.

Go in confident. You've earned it.

Good luck. You've got this.